Music Genre Classification using Spectrograms with Advanced Machine Learning

Introduction

In the realm of music, genre serves as a fundamental descriptor, shaping listeners' expectations and influencing their preferences. With the vast amount of music available online, the ability to automatically classify music by genre is not only a challenging task but also a necessity for music streaming services, recommendation systems, and digital libraries. The goal of this project is to leverage the capabilities of deep learning to accurately classify music genres using the GTZAN dataset, which includes a variety of unstructured audio data, thus highlighting the strengths of these advanced machine learning techniques.

Motivation

The motivation behind this project stems from a fascination with both music and the transformative power of advanced machine learning technologies. Music genre classification poses unique challenges due to the complexity of audio data and the subjective nature of genre categorization. By undertaking this project, I aim to explore the intersection of music and machine learning, applying theoretical knowledge in a practical, impactful manner. Furthermore, this project provides a platform to delve into the nuances of audio signal processing and to understand how advanced machine learning can be applied to real-world data.

Dataset

The GTZAN dataset, which is publicly available on Kaggle, consists of 1000 audio tracks evenly distributed across 10 genres:

- Blues
- Classical
- Country
- Disco
- Hip-hop
- Jazz
- Metal
- Pop
- Reggae
- Rock

Each track is 30 seconds long, providing a rich set of data for analysis. This dataset was chosen for its diversity in music genres and its balanced nature, which facilitates an unbiased approach to the classification task.

Due to the extensive size of the dataset, it was kept on a Google Drive repository and was mounted to Google Colab while analyzing. The Google Colab repository can be accessed from the following link:
https://drive.google.com/drive/folders/1f9XfO_Vd1sErp0KpUnnjF5CcgmcKh6wk?usp=sharing

Exploratory Data Analysis

In the exploratory phase of the project, an in-depth analysis of the GTZAN dataset's audio tracks was conducted. The equal representation of genres, with 100 tracks each, ensures a well-balanced foundation for training the models, minimizing the potential for bias.

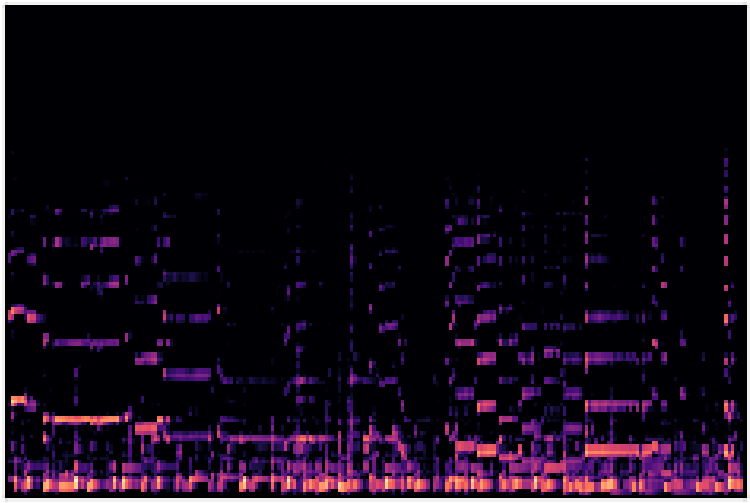


Fig 01: Spectrogram of a Sample Track

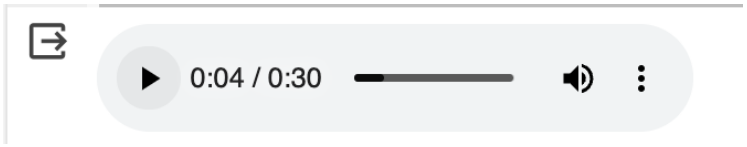


Fig 02: Audio output of the Sample Track

I employed LibROSA, a comprehensive library for audio analysis, to process a sample track by trimming silences and extracting a spectrogram. This visualization process translated the audio file into a rich, visual format, revealing rhythmic complexities and harmonic textures unique to each genre. The Mel-scale spectrogram, resonating with human auditory perception, provided detailed insights into the specific characteristics of musical genres, highlighting sustained notes in classical tracks and dense, low-frequency energies in hip-hop beats.
Listening to the tracks added an auditory dimension to the EDA, allowing us to experience the data as both analysts and listeners. This dual approach grounded the subsequent feature extraction and model training in a thorough understanding of the dataset.

```
Training: {'blues': 80, 'classical': 80, 'country': 80, 'disco': 80, 'hiphop': 80, 'jazz': 80, 'metal': 80, 'pop': 80, 'reggae': 80, 'rock': 80}
Validation: {'blues': 9, 'classical': 9, 'country': 9, 'disco': 9, 'hiphop': 9, 'jazz': 9, 'metal': 9, 'pop': 9, 'reggae': 9, 'rock': 9}
Test: {'blues': 10, 'classical': 10, 'country': 10, 'disco': 10, 'hiphop': 10, 'jazz': 10, 'metal': 10, 'pop': 10, 'reggae': 10, 'rock': 10}
```

Fig 03: Data split into Train, Validation and Test Set

I then split the dataset into training, validation, and testing sets, following a ratio of 80%, 9%, and 10%, respectively. This ensured that each set was representative of the overall diversity in music genres.
To prepare for model training, I established a preprocessing pipeline. This included mapping genre labels to numerical values, converting images to tensors, and normalizing their pixel values. The *prep* function further optimized the datasets with caching and prefetching, creating efficient, batched sets for each stage of model evaluation.

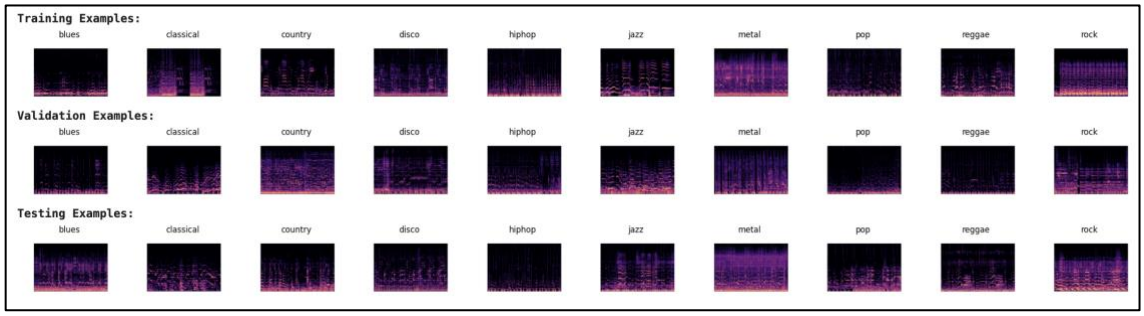


Fig 04: Spectrograms of Different Genre (split into Train, Validation and Test set)

The final step in the EDA was to visually confirm the quality and balance of the datasets. The displayed spectrograms, one from each genre, serve as vibrant indicators of the diverse musical structures I aim to analyze with the deep learning model. This visual mosaic, as shown in the figure above, not only confirms the datasets' readiness but also encapsulates the rich tapestry of genres I will explore.

Following the exploratory data analysis, I transitioned to the development of the deep learning model. The EDA provided crucial insights into the acoustic diversity of the dataset, which informed the architecture and design of the neural network.

Model Development

With the features prepared, the next step was to build and train deep learning models. Here, two different approaches were explored to construct deep learning models capable of genre classification, leveraging the pre-processed spectrogram images.

InceptionV3 as a Feature Extractor

The first approach utilized the InceptionV3 model, known for its high performance in image classification tasks. This model, pre-trained on the ImageNet dataset, was adapted to the task by setting its top layers to non-trainable and appending a new set of layers specifically designed for the classification purpose.

I augmented InceptionV3 with a sequence of operations starting with flattening the output to a one-dimensional array. Batch normalization was applied to standardize the inputs to the subsequent layers, promoting faster convergence. I then added a series of dense layers interspersed with dropout layers. The dropout rate of 0.3 helped mitigate overfitting by randomly setting a portion of the input units to zero during training. The final layer, a dense layer with a softmax activation, outputs the probability distribution over the ten genre classes.

This transfer learning approach allows us to capitalize on the rich feature representations learned by InceptionV3 while customizing the model for the specific task. The choice of a low learning rate in the Stochastic Gradient Descent (SGD) optimizer encourages fine-tuning adjustments to the pre-trained weights without dramatic changes.

Layer (type)	Output Shape	Param #
inception_v3 (Functional)	(None, 2048)	21802784
flatten (Flatten)	(None, 2048)	0
batch_normalization_94 (BatchNormalization)	(None, 2048)	8192
dense (Dense)	(None, 512)	1049088
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32896
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 10)	1290

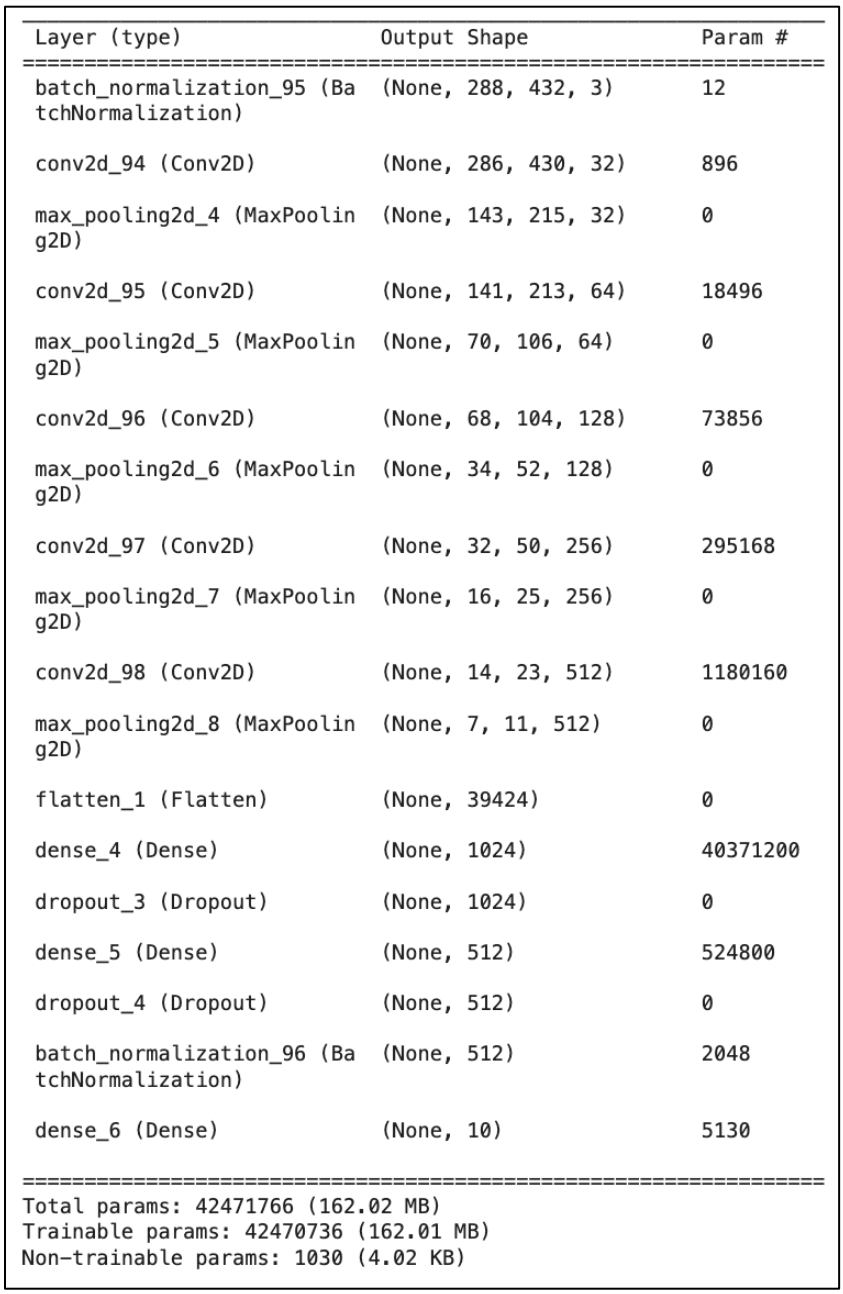
Total params: 23025578 (87.84 MB)
Trainable params: 1218698 (4.65 MB)
Non-trainable params: 21806880 (83.19 MB)

Fig 05: Architecture Summary of Inception_v3

The summary provides a comprehensive view of the sequential model, detailing the layer-wise parameters and their trainable or non-trainable status, underlining the strategic freezing of the InceptionV3 base to preserve its valuable features.

Custom CNN Architecture

In contrast to leveraging a pre-built architecture, I also developed a custom CNN from scratch. The model starts with a batch normalization layer applied to the input, ensuring the initial data is normalized. Following this, I constructed a stack of convolutional layers, each followed by a max-pooling layer. The convolutional layers capture local patterns within the spectrograms, while the pooling layers reduce dimensionality and computational load, extracting the most salient features. After the convolutional base, the model transitions to fully connected layers. I drastically increase the neuron count in the first dense layer to 1024, providing the model with ample capacity to learn from the complex data. Subsequent dropout layers with a higher rate of 0.5 increase the robustness against overfitting. Another batch normalization layer before the final classification layer ensures that the inputs are well-scaled.



Layer (type)	Output Shape	Param #
batch_normalization_95 (BatchNormalization)	(None, 288, 432, 3)	12
conv2d_94 (Conv2D)	(None, 286, 430, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 143, 215, 32)	0
conv2d_95 (Conv2D)	(None, 141, 213, 64)	18496
max_pooling2d_5 (MaxPooling2D)	(None, 70, 106, 64)	0
conv2d_96 (Conv2D)	(None, 68, 104, 128)	73856
max_pooling2d_6 (MaxPooling2D)	(None, 34, 52, 128)	0
conv2d_97 (Conv2D)	(None, 32, 50, 256)	295168
max_pooling2d_7 (MaxPooling2D)	(None, 16, 25, 256)	0
conv2d_98 (Conv2D)	(None, 14, 23, 512)	1180160
max_pooling2d_8 (MaxPooling2D)	(None, 7, 11, 512)	0
flatten_1 (Flatten)	(None, 39424)	0
dense_4 (Dense)	(None, 1024)	40371200
dropout_3 (Dropout)	(None, 1024)	0
dense_5 (Dense)	(None, 512)	524800
dropout_4 (Dropout)	(None, 512)	0
batch_normalization_96 (BatchNormalization)	(None, 512)	2048
dense_6 (Dense)	(None, 10)	5130

Total params: 42471766 (162.02 MB)
Trainable params: 42470736 (162.01 MB)
Non-trainable params: 1030 (4.02 KB)

Fig 06: Architecture Summary of Custom CNN

The custom CNN is a demonstration of a model built from the ground up, tailored to the unique requirements of audio spectrogram classification. The compilation of this model uses the same loss function but with a slightly higher learning rate, recognizing the need for more significant updates due to the random initialization of weights.

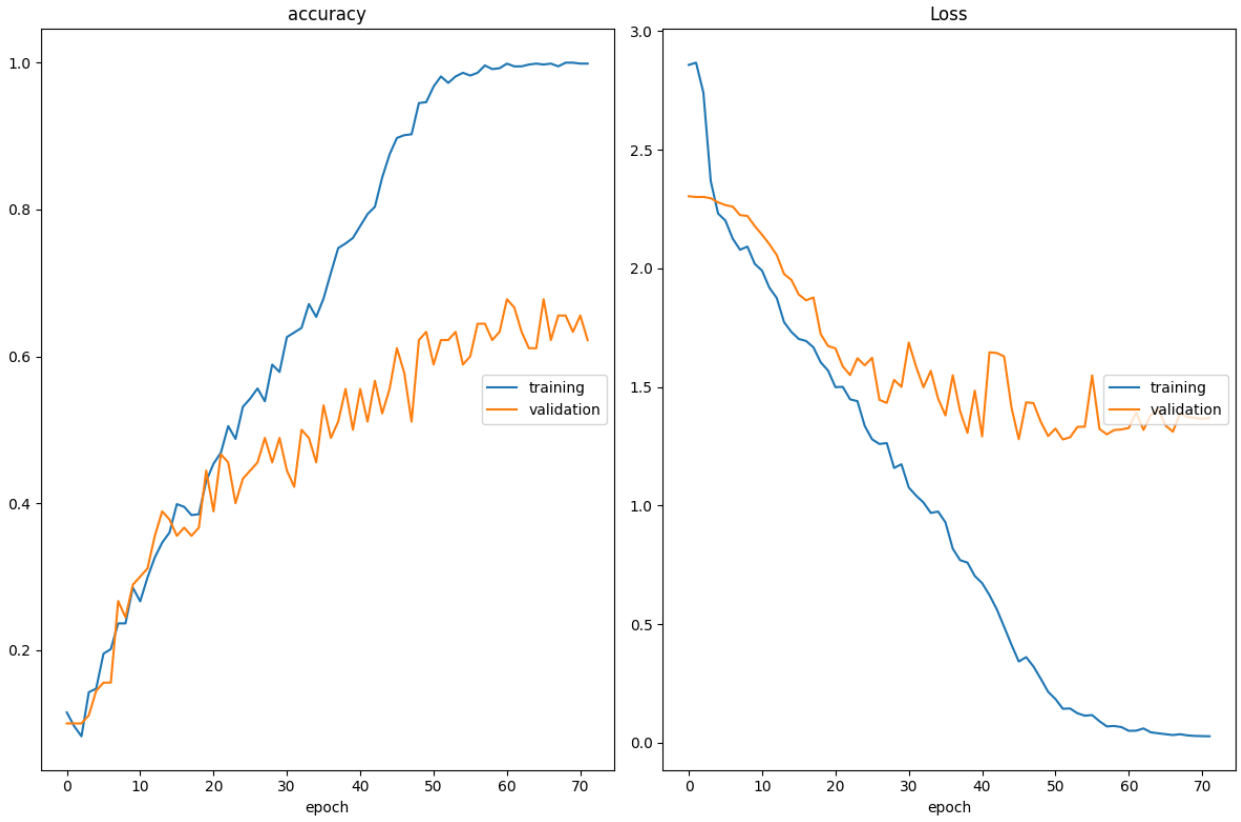


Fig 07: Accuracy and Loss plots on Training and Validation Data

In the training of the custom CNN model, I implemented a systematic and informed process aimed at achieving the best possible accuracy for music genre classification. The model training was set to run for a maximum of 500 epochs, with setRandom() ensuring reproducibility of results through controlled randomization.

The training progression was meticulously tracked using the PlotLossesKeras() callback, which provided real-time plotting of accuracy and loss metrics for both training and validation datasets. This visual feedback is crucial for monitoring the model's learning curve and making any necessary adjustments to the training process.

The training accuracy reached near-perfection, indicating the model's high proficiency on the training data. However, the validation accuracy plateaued at around 62.2%, suggesting that while the model has learned the training data well, it may not generalize as effectively to unseen data.

A similar trend is observed in the loss plots, where the training loss decreased to an admirable 0.028, reflecting the model's confidence in its predictions on the training set. Meanwhile, the validation loss achieved a minimum of 1.279, with some fluctuations, ending slightly higher at 1.370. The discrepancy between training and validation loss could signify overfitting, a common challenge in machine learning where the model learns patterns specific to the training data that do not generalize to other data.

Model Performance

The performance comparison of the Transfer and Custom CNN models is strikingly revealed through their confusion matrices.

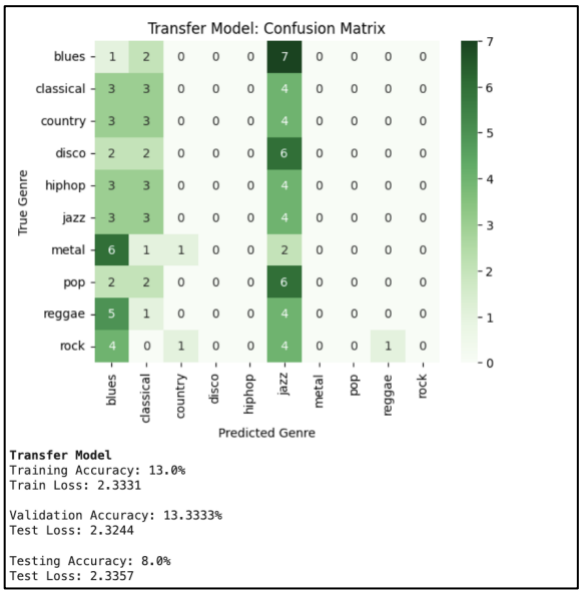


Fig 08: Confusion Matrix of Transfer Model

The Transfer Model, adapted from InceptionV3, exhibits a strong bias towards the 'blues' genre, resulting in low training and testing accuracies of 13.0% and 8.0%, respectively. This suggests a significant shortfall in the model's ability to differentiate between genres.

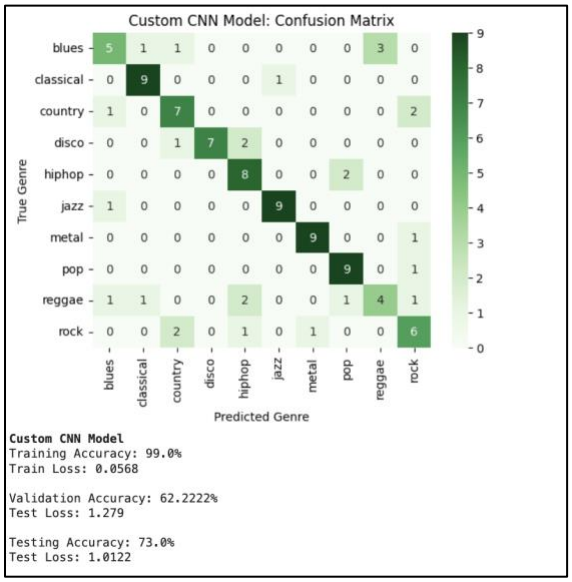


Fig 09: Confusion Matrix of Custom CNN Model

In contrast, the Custom CNN model demonstrates a more nuanced understanding of the dataset, with a more even spread of predictions across genres and a notably higher testing accuracy of 73.0%. It successfully captured the distinct features of certain genres like 'classical' and 'hip-hop', suggesting a better fit for the task at hand.

These outcomes emphasize the Custom CNN model's superior capability to generalize and its potential suitability for complex classification tasks like music genre identification.

Testing:
To provide a practical demonstration of the Custom CNN model's predictive capabilities, a single audio file from the test dataset was randomly selected for genre classification. The audio file's corresponding spectrogram was processed- decoded, resized, and normalized, to match the model's input requirements.

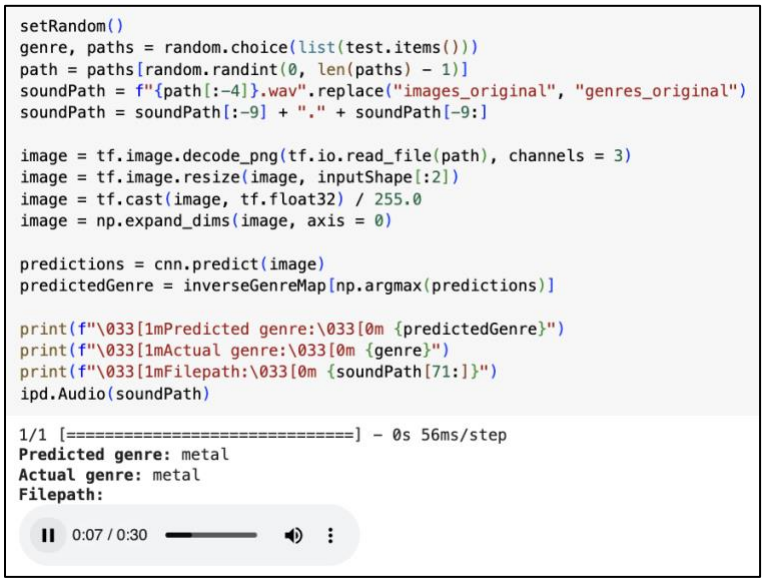


Fig 10: Model Testing

Upon prediction, the model identified the genre of the selected track as 'metal,' which aligned with the actual genre, showcasing the model's proficiency in making accurate predictions for individual samples. This successful prediction illustrates the model's learned ability to discern the characteristics of the 'metal' genre from the audio spectrogram.
The outcome, along with the option to listen to the audio, offers an interactive and conclusive way to verify the model's performance, providing both a quantitative and qualitative assessment of its classification accuracy.

Conclusion
The project successfully demonstrated the application of advanced machine learning to music genre classification, achieving respectable accuracy and providing a foundation for further research. The experience gained from this project is invaluable, particularly in understanding how to handle unstructured audio data with deep learning techniques.

Reflections
This project was a profound learning experience, bridging theoretical knowledge with practical application. It not only reinforced my understanding of deep learning and its potential but also ignited a deeper interest in the field of audio signal processing. As I move forward, the insights gained from this endeavor will undoubtedly influence my approach to future machine learning projects and applications.